

PROYECTO:

PIDDEF 0322

Dirección del proyecto: *Igor PRARIO*

Co-Dirección del proyecto: *Patricio BOS*

Desarrollo de un sistema embebido de adquisición de datos y telemetría satelital para boya instrumental EAMMRA.

Patricio BOS e Igor PRARIO.

División Acústica Submarina

Informe Técnico AS 01/26

Junio 2026



JEIV
JEATURA DE
INVESTIGACIÓN Y DESARROLLO
DE LA ARMADA



UNIDEF
UNIDAD EJECUTORA DE
INVESTIGACIÓN Y DESARROLLO
ESTRATÉGICOS PARA LA DEFENSA
CONICET/MINDEF

Integración de subsistemas para el prototipo de EAMMRA.

Mariano CINQUINI, Patricio BOS, Igor PRARIO y Rui MARQUES ROJO

RESUMEN

En este informe técnico se presenta la integración de cinco subsistemas para el prototipo de Estación Autónoma Marítima para el Monitoreo de Ruido Ambiente (EAMMRA), en el marco del proyecto de I+D PIDDEF 32/14 del Ministerio de Defensa. Los subsistemas que se integran son: alimentación, control, procesamiento, adquisición y sensores. Se documentan sus principales características y se describen las interfaces físicas y lógicas entre subsistemas. Asimismo, se hace una descripción funcional del ciclo de trabajo de la estación a cargo de los subsistemas de control y procesamiento. Finalmente, se incluyen pruebas preliminares de laboratorio para evaluar el desempeño del sistema integrado en su conjunto.

ABSTRACT

In this technical report the integration of five subsystems for the prototype of the Autonomous Maritime Station for Environmental Noise Monitoring (EAMMRA) is presented, within the framework of the R&D project PIDDEF 32/14 of the Ministry of Defense. The integrated subsystems are: power, control, processing, acquisition and sensors. Their main characteristics are documented and the physical and logical interfaces between subsystems are described. Likewise, a functional description of the work cycle of the station in charge of the control and processing subsystems is made. Finally, preliminary laboratory tests are included to evaluate the performance of the integrated system as a whole.

Este trabajo es parte del Proyecto: “EAMMRA”,
del programa PIDDEF del Ministerio de Defensa,
vinculado al Proyecto “Diseño y desarrollo de un prototipo de Módulo Autónomo
para Mediciones de Ruido Ambiente del océano”
de la Armada Argentina,
que se lleva a cabo en la División Acústica Submarina
de la Dirección de Investigación de la Armada (DIIV) y
la Unidad Ejecutora de Investigación y Desarrollo Estratégicos
para la Defensa (UNIDEF), dependiente de CONICET/MinDef.

AGRADECIMIENTOS

Se hace explícito el agradecimiento a:



Mariano CINQUINI

ÍNDICE

I.	Introducción General	7
I.1	Antecedentes	7
I.2	Contexto	8
I.3	Motivación	8
I.4	Propuesta de Valor y Objetivos	9
II.	Introducción Específica	12
II.1	Descripción del Hardware	12
II.2	Herramientas y Tecnologías de Software	14
II.3	Arquitectura General del Sistema	16
II.4	Requerimientos Funcionales y No Funcionales	17
III.	Diseño e Implementación	19
III.1	Arquitectura del sistema	19
III.2	Proceso principal y enrutamiento de eventos	19
III.3	Scheduler y gestión de tareas	19
III.4	Módulos de sensores	19
III.4.a	Sensores ambientales (AHT10, MPU6050)	19
III.4.b	Sensores meteorológicos y de posición	19
III.4.c	Procesamiento de audio	19
III.4.d	Gestión de energía	19
III.5	Telemetría satelital (Iridium SBD)	19
III.6	Protocolos y formatos de datos	19
III.7	Almacenamiento y políticas de persistencia	19
IV.	Pruebas y Ensayos	19
IV.1	Pruebas unitarias y de integración	19
IV.2	Tests de bajo nivel y simulaciones (mocks)	19
IV.3	Ensayos operacionales en laboratorio	19
IV.4	Pruebas de campo y telemetría	20
IV.5	Resultados obtenidos	20
V.	Conclusiones y Trabajo Futuro	20
V.1	Conclusiones	20
V.2	Logros alcanzados	20
V.3	Dificultades encontradas y lecciones aprendidas	20
V.4	Trabajo futuro	20
V.4.a	Mejoras propuestas	20
V.4.b	Nuevas funcionalidades	20
V.4.c	Escalabilidad y mantenimiento del sistema	20
	REFERENCIAS	21

GLOSARIO DE SIGLAS

AIS	<i>Automatic Identification System</i>
ARA	Armada Argentina
CONICET	Consejo Nacional de Investigaciones Científicas y Técnicas
DIIV	Dirección de Investigación de la Armada
GSJ	Golfo San Jorge
FSM	<i>Finite State Machine</i>
GNU	<i>GNU is Not Unix</i>
GPS	<i>Global Position System</i>
I2C	<i>Inter-Integrated Circuit</i>
I+D	Investigación y Desarrollo
JEIV	Jefatura de Investigación y Desarrollo de la Armada
JSON	<i>JavaScript Object Notation</i>
JSONL	JSON Lines
MinDef	Ministerio de Defensa
MEF	Máquina de Estados Finitos
MPPT	<i>Maximum Power Point Tracking</i>
NL	Nivel de Ruido
NL_a	Ruido Ambiente marino propiamente dicho
NL_p	Ruido Propio asociado al sistema de medición
PAM	<i>Passive Acoustic Monitoring</i>
PIDDEF	Proyecto de Investigación y Desarrollo para la Defensa
SDB	<i>Short Data Burst</i>
SHN	Servicio de Hidrografía Naval
SONAR	<i>SOund Navigation And Range</i>
TL	<i>Transmission Loss</i> (Pérdidas por Transmisión acústica)
TS	<i>Target Strength</i> (Fuerza de Blanco)
UNIDEF	Unidad Ejecutora de Investigación y Desarrollo Estratégicos para la Defensa

I. Introducción General

I.1. Antecedentes

El presente informe técnico documenta el desarrollo del sistema embebido de control, adquisición, supervisión y telemetría de la boya instrumental EAMMRA, concebida para la medición autónoma de variables ambientales, meteorológicas, de posicionamiento y acústicas en ambiente marítimo. El trabajo constituye una nueva etapa de maduración tecnológica respecto de desarrollos previos orientados al control de una estación autónoma marítima de monitoreo de ruido ambiente submarino, en los cuales se habían establecido las bases conceptuales, funcionales y metodológicas para una arquitectura embebida modular Bos *et al.* [2016]; Bos [2018].

La línea de desarrollo original estuvo centrada en el problema de medir, registrar y analizar el Nivel de Ruido submarino, NL , parámetro de relevancia en acústica submarina, escucha pasiva, caracterización ambiental y aplicaciones vinculadas a modelos de propagación y predicción SONAR. En ese marco, se distinguieron componentes fundamentales del ruido en el mar: el ruido ambiente propiamente dicho, NLa , asociado al fondo acústico natural y antrópico; el ruido propio, NLp , generado por el sistema de medición y su plataforma; y el ruido radiado por fuentes identificables. Esta distinción resulta esencial para el diseño de sistemas de medición, ya que condiciona la selección de sensores, la electrónica de acondicionamiento, los rangos dinámicos requeridos y las estrategias de procesamiento de datos Bos *et al.* [2016].

Los trabajos iniciales sobre EAMMRA permitieron avanzar desde una formulación conceptual hacia una arquitectura de estación autónoma, identificando subsistemas de sensado, adquisición, procesamiento, control, almacenamiento y alimentación. En particular, la memoria desarrollada en el marco de la Maestría en Sistemas Embebidos documentó una primera implementación de referencia para el sistema de control, con énfasis en firmware modular, programación cooperativa, trazabilidad de requerimientos, pruebas unitarias, pruebas funcionales y validación progresiva del comportamiento del sistema Bos [2018]. Aquella implementación constituyó una base metodológica importante, aunque limitada por el alcance del hardware disponible y por el grado de integración de la plataforma en esa etapa.

En paralelo, otros desarrollos abordaron aspectos complementarios del sistema completo: el diseño de la boya prototipo como plataforma física de superficie Ezcurra *et al.* [2019], la integración de subsistemas electrónicos y de sensado Cinquini *et al.* [2019], y el desarrollo de etapas analógicas específicas, como el preamplificador para hidrófonos Cinquini *et al.* [2018]. Estos antecedentes permitieron consolidar conocimiento técnico sobre la operación en campo, la integración de sensores heterogéneos, las restricciones de consumo, las necesidades de robustez mecánica y electrónica, y la importancia de contar con procedimientos verificables para adquisición, almacenamiento y recuperación de datos.

La etapa actual, correspondiente al PIDDEF 0322, representa una evolución sustantiva respecto de esos antecedentes. El sistema ya no se limita a una prueba conceptual o a un prototipo parcial de control, sino que se orienta a documentar el sistema embebido que controla una boya EAMMRA operativa. Esta nueva versión incorpora una computadora de procesamiento distinta, mayor cantidad de hardware integrado, sensores adicionales, telemetría satelital, gestión energética más elaborada, políticas de persistencia robustas y una arquitectura de software con mayor nivel de modularidad, verificabilidad y tolerancia a fallos.

El nuevo sistema se desarrolla sobre una arquitectura orientada a operación real, con integración física en boya, interacción con subsistemas de energía, comunicaciones remotas, sensores distribuidos, almacenamiento persistente y mecanismos de recuperación ante fallas. Por lo tanto, el foco del informe se desplaza desde la demostración de conceptos de control embebido hacia la documentación de un sistema operativo, integrado y ensayable.

I.2. Contexto

El sistema EAMMRA se inscribe dentro del desarrollo de plataformas autónomas de observación marítima, cuyo propósito es adquirir información en sitios donde la presencia continua de operadores resulta impracticable, costosa o técnicamente inconveniente. Una boya instrumental de estas características debe operar en un entorno severo, con exposición a humedad, salinidad, variaciones térmicas, vibraciones, oleaje, viento, restricciones energéticas y ventanas de mantenimiento limitadas. Estas condiciones imponen exigencias simultáneas sobre el diseño mecánico, la electrónica, el software de control, la alimentación y la confiabilidad general del sistema.

Desde el punto de vista funcional, la boya debe coordinar múltiples subsistemas heterogéneos. Entre ellos se incluyen sensores ambientales, sensores meteorológicos, receptores de posicionamiento, adquisición de audio, almacenamiento local, supervisión de energía y comunicaciones remotas. Cada uno de estos subsistemas posee tiempos de respuesta, protocolos, consumos, modos de falla y requerimientos de inicialización particulares. La función del sistema embebido es abstraer esa complejidad y organizarla en un ciclo de operación coherente, verificable y tolerante a fallas.

El desarrollo actual adopta una arquitectura de software moderna basada en Python 3, Máquinas de Estados Finitos, un router de eventos y un scheduler central. Esta organización permite representar explícitamente los estados operativos de cada módulo, las transiciones válidas, las condiciones de error, los eventos generados por sensores y tareas, y la ejecución planificada de acciones periódicas o condicionadas. La utilización de este enfoque favorece la separación de responsabilidades, reduce el acoplamiento entre módulos y facilita la simulación de escenarios mediante componentes sustitutos o *mocks*.

La incorporación de telemetría satelital mediante Iridium SBD constituye una mejora relevante respecto de configuraciones basadas exclusivamente en almacenamiento local. Si bien el almacenamiento en la boya continúa siendo indispensable para preservar registros completos o de mayor volumen, la telemetría permite transmitir información resumida, estados de salud, eventos críticos y datos seleccionados sin necesidad de recuperar físicamente la plataforma. Esta capacidad incrementa el valor operativo de la boya, permite realizar seguimiento remoto del despliegue y habilita estrategias de diagnóstico temprano ante comportamientos anómalos.

Asimismo, el sistema de alimentación con paneles solares y gestión inteligente de energía introduce una dimensión crítica en el diseño embebido. En una plataforma autónoma, la energía disponible condiciona el ciclo de trabajo, la frecuencia de adquisición, la disponibilidad de comunicaciones, los períodos de bajo consumo y las políticas de protección de los subsistemas. Por ello, el software de control debe actuar no sólo como coordinador lógico de tareas, sino también como administrador del presupuesto energético, priorizando funciones, reduciendo consumos innecesarios y preservando la continuidad operativa.

I.3. Motivación

La motivación principal del proyecto es disponer de una boya instrumental autónoma, robusta y operativa para adquirir datos relevantes del ambiente marítimo y transmitir información esencial en forma remota. La medición del ruido submarino y de variables asociadas permite construir series temporales de valor científico y técnico, mejorar la caracterización de escenarios acústicos y correlacionar la información hidroacústica con condiciones ambientales, meteorológicas y de posición.

En sistemas de este tipo, el dato acústico aislado resulta insuficiente si no se encuentra acompañado por información contextual. La interpretación de un registro hidroacústico requiere conocer, entre otros factores, la fecha y hora de adquisición, la ubicación de la plataforma, el estado de la boya, las condiciones ambientales, el viento, la temperatura y la validez del ciclo de medición. Por esta razón, el sistema

embebido debe concebirse como una infraestructura de adquisición multi-sensor, y no únicamente como un controlador de un dispositivo acústico.

La motivación tecnológica se vincula con la necesidad de incrementar el nivel de maduración del sistema respecto de versiones anteriores. En una etapa inicial, el objetivo principal era demostrar la factibilidad de una arquitectura de control modular y ensayable. En la etapa actual, el objetivo es controlar una plataforma real, con mayor cantidad de hardware, mayores exigencias de integración, comunicación remota y tolerancia a fallos. Esto requiere pasar de una lógica de prototipo a una lógica de sistema operativo en campo, donde las decisiones de diseño deben contemplar degradación controlada, reinicio seguro, persistencia de datos, diagnóstico y mantenibilidad.

El uso de Máquinas de Estados Finitos responde a la necesidad de hacer explícito el comportamiento del sistema ante condiciones normales y anómalas. Cada módulo puede describirse en términos de estados, eventos, transiciones y acciones, lo que simplifica el análisis de comportamiento, la implementación de pruebas y la documentación técnica. A su vez, el router de eventos permite desacoplar productores y consumidores de información, mientras que el scheduler central organiza la ejecución temporal de tareas periódicas, diferidas o condicionadas por estado.

Otra motivación central es la necesidad de verificabilidad. Una boya operativa no puede depender de supuestos implícitos ni de secuencias manuales difíciles de reproducir. El sistema debe poder probarse en laboratorio, simularse parcialmente, integrarse por etapas y ensayarse en condiciones progresivamente más representativas. Para ello se emplean componentes simulados, pruebas unitarias, pruebas de integración, registros de ejecución y políticas de validación orientadas a comprobar tanto el funcionamiento nominal como la respuesta frente a fallas previsibles.

I.4. Propuesta de Valor y Objetivos

La propuesta de valor del presente desarrollo consiste en consolidar un sistema embebido integral para la boya EAMMRA, capaz de coordinar adquisición multi-sensor, almacenamiento local, supervisión energética, procesamiento básico, registro de eventos y telemetría satelital. El sistema busca proveer una plataforma de control confiable, modular y extensible, apta para operar sobre hardware real y para ser mantenida, ensayada y ampliada en futuras etapas del proyecto.

El objetivo general del informe es documentar el diseño, la implementación y la validación del sistema embebido que controla la boya EAMMRA como entregable principal del PIDDEF 0322. Esta documentación abarca la arquitectura de software, la organización de tareas, los módulos de adquisición, los mecanismos de enrutamiento de eventos, las políticas de persistencia, la telemetría satelital, la gestión energética y los criterios de prueba aplicados al sistema.

Como objetivos específicos se establecen los siguientes:

- Describir la evolución del sistema respecto de desarrollos previos de control embebido para EAMMRA, identificando los cambios de arquitectura, plataforma de procesamiento, hardware integrado y alcance funcional.
- Presentar una arquitectura de software modular basada en Máquinas de Estados Finitos, router de eventos y scheduler central, orientada a operación autónoma y tolerancia a fallos.
- Documentar la integración de sensores ambientales, meteorológicos, de posicionamiento y audio, junto con sus interfaces, ciclos de adquisición y criterios de validación.
- Describir las políticas de almacenamiento persistente y guardado seguro, orientadas a preservar la integridad de los datos ante interrupciones, reinicios o fallas parciales.

- Documentar la integración de telemetría satelital mediante Iridium SBD para transmisión remota de datos, eventos y estados relevantes de la boya.
- Presentar la estrategia general de gestión energética, considerando alimentación solar, consumo de subsistemas, modos de operación y criterios de protección.
- Establecer una base de verificación y validación mediante pruebas unitarias, pruebas de integración, simulaciones, *mocks*, ensayos de bajo nivel y pruebas operacionales.
- Generar documentación técnica suficiente para mantenimiento, auditoría, continuidad de desarrollo y transferencia del sistema a nuevas configuraciones de boya instrumental.

El resultado esperado es una descripción técnica completa del sistema embebido de control de EAMMRA, con nivel de detalle suficiente para comprender su arquitectura, justificar sus decisiones de diseño, reproducir sus ensayos principales y orientar futuras ampliaciones. La madurez alcanzada por la plataforma permite considerar este desarrollo no sólo como una implementación particular, sino como una base reutilizable para sistemas autónomos de observación marítima con adquisición multi-sensor y comunicaciones remotas.

En síntesis, este informe documenta el paso desde una arquitectura embebida demostrativa hacia un sistema de control integrado en una boya operativa. La combinación de adquisición multi-sensor, software modular, telemetría satelital, gestión energética, almacenamiento seguro y pruebas sistemáticas representa un avance significativo en la evolución de EAMMRA y constituye el núcleo técnico del entregable documentado en el marco del PIDDEF 0322.

Los requerimientos funcionales del sistema surgen de la necesidad de controlar una boya instrumental autónoma con múltiples subsistemas integrados. En términos generales, el sistema debe ser capaz de adquirir datos, validarlos, almacenarlos, procesarlos parcialmente y transmitir información seleccionada en forma remota.

Los requerimientos funcionales principales son:

- Adquirir mediciones ambientales de temperatura y humedad.
- Adquirir mediciones inerciales de aceleración y velocidad angular.
- Adquirir información de posicionamiento, tiempo y datos de navegación mediante AIS/GPS.
- Adquirir mediciones meteorológicas de velocidad y dirección del viento.
- Relevar variables eléctricas del sistema de alimentación, incluyendo parámetros asociados a panel solar, carga y batería.
- Ejecutar adquisiciones acústicas mediante placa de audio USB y almacenar archivos WAV con metadatos asociados.
- Procesar archivos de audio para obtener productos compactos compatibles con almacenamiento local y telemetría satelital.
- Registrar mediciones en archivos persistentes con formato estructurado y unidades explícitas.
- Registrar logs de ejecución, eventos relevantes, errores, advertencias y condiciones de diagnóstico.
- Transmitir mensajes de estado de sistema mediante Iridium SBD.
- Transmitir resultados seleccionados de procesamiento acústico mediante Iridium SBD.

- Ejecutar tareas periódicas mediante un scheduler central configurable.
- Representar el comportamiento de cada módulo mediante Máquinas de Estados Finitos.
- Permitir ejecución con hardware real, con todos los módulos simulados o con simulación parcial de módulos seleccionados.
- Permitir pruebas automatizadas sin hardware y pruebas específicas contra hardware real.
- Mantener archivos de configuración editables para adaptar intervalos, rutas, parámetros de adquisición, habilitación de telemetría y uso de *mocks*.

Los requerimientos no funcionales responden a las restricciones propias de una plataforma autónoma embarcada. Entre ellos se destacan:

- **Autonomía operativa:** el sistema debe ejecutar ciclos de adquisición, almacenamiento y transmisión sin intervención humana continua.
- **Robustez:** el sistema debe tolerar fallas parciales de módulos, errores de lectura, indisponibilidad transitoria de periféricos y condiciones de comunicación no ideales.
- **Persistencia segura:** los datos adquiridos no deben perderse por fallas evitables de software, configuraciones ambiguas o políticas agresivas de liberación de espacio.
- **Bajo consumo relativo:** el software debe permitir ciclos de adquisición y transmisión compatibles con el presupuesto energético disponible.
- **Mantenibilidad:** la separación entre bajo nivel, FSM, Router y scheduler debe facilitar la modificación de módulos individuales sin afectar la arquitectura completa.
- **Extensibilidad:** la arquitectura debe permitir la incorporación futura de nuevos sensores, nuevos productos de procesamiento o nuevos formatos de telemetría.
- **Verificabilidad:** el comportamiento del sistema debe poder probarse mediante tests automatizados, ejecución con *mocks*, ensayos de bajo nivel y pruebas de integración.
- **Trazabilidad:** las mediciones, eventos, configuraciones y resultados de prueba deben poder relacionarse con módulos, horarios de ejecución y condiciones operativas.
- **Degradación controlada:** ante una falla, el sistema debe registrar la condición, conservar datos previos, evitar acciones inseguras y continuar operando en la medida de lo posible.
- **Diagnóstico remoto:** los mensajes satelitales deben permitir inferir el estado general de la boya, incluyendo salud de módulos, almacenamiento, energía y continuidad operativa.

En conjunto, estos requerimientos justifican la arquitectura adoptada. La combinación de módulos de bajo nivel, FSM, Router, scheduler central, almacenamiento estructurado, telemetría binaria, pruebas automatizadas y soporte de simulación permite abordar la complejidad de una boya operativa sin concentrar toda la lógica en un bloque monolítico. La Introducción Específica presentada en esta sección establece, por lo tanto, el marco técnico necesario para comprender las decisiones de diseño e implementación desarrolladas en las secciones siguientes.

II. Introducción Específica

II.1. Descripción del Hardware

La boya EAMMRA integra un conjunto heterogéneo de subsistemas de sensado, procesamiento, almacenamiento, alimentación y comunicaciones, coordinados por una computadora principal embarcada. A diferencia de las etapas previas del desarrollo, el sistema documentado en este informe corresponde a una configuración operativa de mayor madurez tecnológica, en la cual el hardware no se limita a una plataforma de ensayo aislada, sino que se encuentra integrado como parte funcional de una boya instrumental.

El núcleo de procesamiento está constituido por una computadora principal de propósito general, de grado industrial marca Kaise, modelo KBox-S-J6412, apta para ejecutar un sistema operativo de tipo GNU/Linux y aplicaciones de control desarrolladas en Python-3. Esta computadora cuenta con un procesador Intel Celeron J6412@2.00GHz de cuatro núcleos y un sistema de refrigeración pasivo sin partes móviles tipo fanless. Posee múltiples interfaces de entrada/salida para conectar los distintos periféricos del sistema, como se puede apreciar en las vistas frontal y posterior del dispositivo, en la figura 1.

Vista frontal de E/S



Vista posterior de E/S



FIG. 1. Vista frontal y trasera de la computadora embarcada KAISE integrada al sistema de control de la boya EAMMRA.

Esta computadora concentra la ejecución de diferentes módulos de software, que se ocupan del enrutamiento de eventos, la planificación de tareas, la adquisición de datos, el registro persistente, el procesamiento básico y la preparación de mensajes de telemetría. La elección de una arquitectura basada en una computadora embarcada permite disponer de recursos de procesamiento, almacenamiento e interfaces superiores a los de una implementación exclusivamente microcontrolada, lo cual resulta especialmente relevante para la adquisición y procesamiento de audio.

El hardware integrado puede organizarse funcionalmente en los siguientes bloques:

- **Sensado ambiental:** incluye sensores de temperatura y humedad, con comunicación digital mediante bus I2C. Estos sensores permiten registrar variables internas de bajo ancho de banda, útiles para la caracterización y el diagnóstico operativo del sistema.

- **Sensado inercial:** incluye una unidad inercial de acelerómetro y giróscopo, también integrada mediante bus I2C. Su función es registrar variables asociadas al movimiento de la boya, las cuales pueden resultar relevantes para interpretar condiciones de operación, eventos mecánicos o correlaciones con registros acústicos.
- **Posicionamiento y datos de navegación:** incluye un receptor AIS/GPS conectado mediante puerto serie. Este subsistema permite disponer de información de posición, fecha, hora y transmite la posición en tiempo real a barcos cercanos y estaciones costeras para prevenir colisiones y mejorar la seguridad marítima.
- **Sensado meteorológico:** incluye un anemómetro ultrasónico, que se comunica por puerto serie, destinado al registro de velocidad y dirección del viento. Esta información es relevante para correlacionar las mediciones acústicas con condiciones de superficie, dado que el viento y el oleaje influyen directamente sobre el ruido ambiente submarino.
- **Supervisión energética:** incluye el controlador tipo MPPT asociado al sistema de generación y almacenamiento de energía, comunicado mediante puerto serie con protocolo Modbus. Este bloque permite relevar variables como tensión de panel, corriente de panel, corriente de carga, tensión de batería, estado de carga y otros parámetros eléctricos necesarios para la gestión operativa de la boya.
- **Adquisición acústica:** incluye una placa de audio profesional USB, dos preamplificadores para hidrófonos de diseño propio, [Cinquini *et al.*, 2018] y dos hidrófonos piezoeléctricos calibrados. Este bloque constituye la etapa de adquisición de señales acústicas de la boya que es la finalidad principal de la boya instrumental EAMMRA.
- **Telemetría satelital:** incluye un módem Iridium, compatible con el servicio SBD. Se utiliza para transmitir mensajes binarios de estado y resultados seleccionados de procesamiento. Este subsistema permite supervisión remota de la boya y envío periódico de información crítica.
- **Almacenamiento local:** incluye medios persistentes destinados a datos crudos, resultados procesados, registros en formato JSONL, logs de ejecución, estados de sistema y archivos de audio crudos. El almacenamiento local se considera el repositorio primario de datos de la boya, mientras que la telemetría satelital se reserva para información resumida y telemetría.

Los manuales de los principales periféricos se encuentran incorporados en la carpeta `docs/` del repositorio del proyecto. Esta documentación se utiliza como referencia técnica para la configuración de interfaces, parámetros eléctricos, protocolos de comunicación, comandos de control y restricciones operativas de los dispositivos integrados Bos [2026b].

La cadena acústica merece una consideración particular. El subsistema de audio ensayado está compuesto por una placa Behringer UMC202HD de dos canales, configurada para adquisición simultánea, y dos preamplificadores para hidrófonos denominados DAS-1 y DAS-2. La placa posee una frecuencia de muestreo de hasta 192 kHz por canal y resolución nominal de 24 bits, mientras que los preamplificadores están diseñados para trabajar con cargas capacitivas del orden de los nF, respuesta extendida hasta 100 kHz, corte en bajas frecuencias e implementación con amplificadores operacionales de bajo ruido LT1792. Estos preamplificadores fueron diseñados ad-hoc considerando el acople de impedancia con hidrófono B&K 8105. La caracterización específica de este equipamiento se documenta en la Nota Técnica AS 02/26, donde se relevaron la respuesta en frecuencia de la placa, la respuesta de los preamplificadores, la figura de ruido propio y el crosstalk entre canales Cinquini [2026].

II.2. Herramientas y Tecnologías de Software

El sistema de control fue desarrollado en Python 3 y se ejecuta sobre la computadora principal embarcada. La adopción de Python responde a la necesidad de contar con un entorno flexible para integración de hardware, ejecución de procesos concurrentes, manejo de archivos, pruebas automatizadas, simulación de periféricos, procesamiento de datos y codificación de mensajes de telemetría.

El código fue desarrollado bajo el sistema de control de versiones distribuido git en un repositorio en la nube github, [Bos, 2026a]. El repositorio integra en una única base de trabajo el código fuente del sistema embebido, los archivos de configuración, los manuales de periféricos, los scripts auxiliares, los datos de soporte y los ensayos automatizados.

El repositorio organiza cada subsistema en dos capas principales. La primera corresponde a los módulos de bajo nivel, identificados con el sufijo `_ll.py`, responsables del acceso directo al hardware, drivers, protocolos o procesamiento específico. La segunda utiliza el modelo de Máquina de Estados Finitos y se identifica con el sufijo `_fsm.py`. Esta capa es responsable de recibir mensajes, ejecutar acciones, gestionar estados internos, informar resultados y reportar condiciones de error. Esta separación permite desacoplar la lógica de control del acceso físico a cada dispositivo.

El proceso principal, implementado en `main.py`, inicializa el sistema, carga configuraciones, instancia los módulos funcionales, lanza las FSM en procesos separados, conecta las colas de comunicación con un enrutador o *router* y activa el planificador de tareas o *scheduler* central. De esta manera, el comportamiento general de la boya queda organizado como un conjunto de módulos independientes que intercambian mensajes mediante una infraestructura común.

Las herramientas de configuración se organizan en archivos JSON. Entre ellos se destacan:

- `configs/config.json`, que contiene parámetros generales de adquisición, procesamiento, directorios de datos, directorios de logs y habilitación de transmisión Iridium.
- `configs/scheduler.json`, que define los intervalos de ejecución para cada subsistema.
- `configs/mock_modules.json`, que permite seleccionar módulos de bajo nivel simulados para ejecuciones repetibles, pruebas sin hardware o ensayos parciales.

El sistema incorpora soporte explícito para *mocks*. Esto permite ejecutar el flujo normal de las FSM sin requerir necesariamente la presencia física de todos los dispositivos. Esta capacidad es fundamental para pruebas de laboratorio, desarrollo incremental, depuración de casos de falla y validación de la lógica de coordinación antes de operar sobre hardware real.

Para la ejecución de pruebas automatizadas se utilizó pytest, un framework de pruebas para Python orientado a la escritura de tests unitarios, funcionales y de integración con una sintaxis simple y legible, pytest Development Team [2026]. A diferencia de enfoques más verbosos, pytest permite expresar verificaciones mediante sentencias `assert` convencionales, descubrir automáticamente archivos y funciones de prueba siguiendo convenciones de nombre, definir contextos reutilizables mediante *fixtures* y organizar subconjuntos de ensayos mediante marcas o configuraciones específicas.

En el presente desarrollo, pytest se empleó como herramienta principal para verificar el comportamiento de módulos individuales, validar la lógica de las Máquinas de Estados Finitos, ejecutar pruebas con componentes simulados y separar ensayos que no requieren hardware de aquellos destinados a periféricos reales. Su incorporación permite mejorar la repetibilidad de los ensayos, reducir regresiones durante el desarrollo y sostener una estrategia de verificación compatible con una arquitectura modular y mantenible.

El registro de datos y eventos se realiza mediante archivos locales. Las mediciones se almacenan típicamente en formato JSONL, con una línea por adquisición y campos con unidades explícitas cuando corresponde. Los logs de ejecución permiten reconstruir la secuencia de eventos, diagnosticar fallas y verificar el comportamiento del sistema durante operación real o simulada.

Entorno de trabajo y dependencias

Para aislar las dependencias del proyecto respecto del sistema operativo base se utilizó un entorno virtual de Python, ubicado convencionalmente en el directorio `.venv`. Este mecanismo permite mantener un intérprete y un conjunto de bibliotecas específicos para la aplicación, lo que reduce interferencias con otros paquetes instalados en el sistema y favorece la reproducibilidad del entorno de ejecución, Python Software Foundation [2026]; Python Packaging Authority [2026].

La instalación del entorno se realiza desde la raíz del repositorio. En primer lugar, se requiere disponer de ciertas dependencias del sistema operativo asociadas al acceso a hardware, interfaces de bajo nivel y adquisición de audio. En particular, el proyecto recomienda instalar los paquetes:

```
sudo apt-get update
sudo apt-get install -y portaudio19-dev python3-dev i2c-tools
```

Luego, el entorno virtual se activa mediante:

```
source .venv/bin/activate
```

y las dependencias de Python se instalan mediante `pip`. Para una instalación orientada exclusivamente a ejecución en runtime, sin herramientas de prueba, se utiliza:

```
python -m pip install -r support/requirements.txt
```

Para desarrollo, verificación y ejecución de pruebas automatizadas, se instala el conjunto ampliado de dependencias:

```
python -m pip install -r support/requirements-dev.txt
```

El archivo `support/requirements.txt` define las bibliotecas necesarias para la operación nominal del sistema. Las versiones se especifican mediante restricciones de compatibilidad, salvo en el caso de `PyAudio`, cuya versión se fija explícitamente:

TABLA 1. Dependencias principales de runtime del sistema.

Biblioteca	Versión requerida	Uso principal
numpy	<code>>=2.0, <3</code>	Cálculo numérico y procesamiento de datos.
pandas	<code>>=2.0, <4</code>	Manejo de datos tabulares y archivos estructurados.
scipy	<code>>=1.14, <2</code>	Procesamiento científico y análisis de señales.
PyAudio	<code>==0.2.14</code>	Interfaz con PortAudio para adquisición de audio.
pyserial	<code>>=3.5, <4</code>	Comunicación con periféricos mediante puertos serie.
smbus2	<code>>=0.5, <1</code>	Comunicación con dispositivos sobre bus I2C.

El archivo `support/requirements-dev.txt` extiende las dependencias de runtime e incorpora herramientas específicas para desarrollo, pruebas y generación de gráficos. En particular, incluye:

TABLA 2. Dependencias adicionales de desarrollo y pruebas.

Biblioteca	Versión requerida	Uso principal
pytest	$\geq 8, < 10$	Ejecución de pruebas automatizadas.
pytest-cov	$\geq 7, < 8$	Medición de cobertura de pruebas.
matplotlib	$\geq 3.8, < 4$	Generación de gráficos y reportes técnicos.

Esta separación entre dependencias de runtime y dependencias de desarrollo permite distinguir el entorno mínimo requerido para operar la boya del entorno ampliado utilizado durante integración, depuración y validación. En una plataforma embebida operativa resulta conveniente mantener el entorno de ejecución tan acotado como sea posible, mientras que en laboratorio se justifica incorporar herramientas adicionales para pruebas, análisis y generación de evidencia.

El uso del entorno virtual también facilita la ejecución explícita de comandos desde el intérprete asociado al proyecto. Por ejemplo, la aplicación principal puede iniciarse con:

```
PYTHONPATH=. .venv/bin/python main.py
```

mientras que la suite de pruebas sin hardware puede ejecutarse mediante:

```
PYTHONPATH=. .venv/bin/python -m pytest -m "not hardware" -q
```

De esta manera, el entorno de trabajo queda definido por tres niveles complementarios: paquetes del sistema operativo necesarios para interfaces físicas, entorno virtual Python para aislamiento de bibliotecas, y archivos `requirements` para reproducir las versiones compatibles de las dependencias del proyecto. Esta organización contribuye a la trazabilidad, repetibilidad y mantenibilidad del software embarcado.

II.3. Arquitectura General del Sistema

La arquitectura general del sistema se basa en una organización modular orientada a eventos. Cada subsistema se representa mediante una FSM que encapsula sus estados, transiciones, acciones y condiciones de error. La comunicación entre módulos se realiza mediante mensajes, y un enrutador (Router) que actúa como intermediario encargado de distribuir eventos entre productores y consumidores de información.

Desde un punto de vista funcional, la arquitectura puede representarse mediante las siguientes capas:

- **Capa de hardware:** comprende sensores, placa de audio, módem satelital, controlador solar, interfaces serie, buses I2C, almacenamiento y dispositivos físicos asociados.
- **Capa de bajo nivel:** implementa el acceso directo a cada dispositivo o servicio específico. Incluye inicialización de interfaces, lectura de sensores, adquisición de audio, comunicación AT/SBD, lectura de controladores de energía y funciones auxiliares de hardware.
- **Capa de FSM:** implementa la lógica de estado de cada módulo. Cada FSM recibe eventos, ejecuta acciones, valida resultados, registra estados y comunica eventos de salida al resto del sistema.
- **Capa de enrutamiento:** centraliza la distribución de mensajes mediante el Router, evitando dependencias directas entre módulos.

- **Capa de planificación:** ejecuta el scheduler central, responsable de disparar adquisiciones periódicas, transmisiones, tareas de mantenimiento y ciclos operativos de acuerdo con la configuración vigente.
- **Capa de persistencia:** administra la escritura de mediciones, logs, estados, archivos de audio, resultados procesados y mensajes preparados para transmisión.
- **Capa de telemetría:** prepara y transmite mensajes binarios mediante Iridium SBD, incluye estados de sistema y resultados seleccionados de procesamiento acústico.

El flujo general de operación comienza con la inicialización de configuraciones, módulos y recursos del sistema. Una vez iniciado el proceso principal, el planificador central dispara eventos de adquisición de acuerdo con intervalos definidos. Los sensores ambientales, meteorológicos, inerciales, de posición y energía generan registros periódicos. Para los ensayos de verificación, la adquisición acústica se ejecuta con una periodicidad menor, debido al mayor volumen de datos generado.

El módulo Behringer realiza la adquisición de audio y genera archivos WAV con metadatos asociados. Al finalizar una adquisición válida, el resultado queda disponible para el módulo AudioProc, que procesa el archivo de audio y genera resultados compactos, persistidos en formato JSON. Estos resultados pueden ser posteriormente seleccionados por el módulo Iridium para su transmisión satelital.

La telemetría Iridium se organiza en ciclos regulares. El sistema puede transmitir mensajes de estado, que resumen la condición operativa de las FSM, los módulos de bajo nivel, la disponibilidad de almacenamiento, el estado energético y el tiempo desde el arranque. También puede transmitir resultados derivados del procesamiento de audio, compactados en formato binario para adecuarse a las restricciones de tamaño del enlace SBD.

El diseño evita que la transmisión satelital dependa directamente de la finalización de una adquisición. En su lugar, el Router registra la disponibilidad del último resultado y el scheduler central gobierna los momentos de transmisión. Esta decisión reduce acoplamientos, simplifica el control temporal del enlace satelital y permite mantener un comportamiento predecible aun cuando alguna tarea se demore, falle o produzca datos incompletos.

La política de almacenamiento de grabaciones de audio es conservadora. El sistema verifica la disponibilidad del directorio de grabaciones, estima el tamaño máximo del archivo WAV a generar y aplica márgenes de seguridad antes de iniciar una nueva adquisición. Si las condiciones de almacenamiento no son aceptables, la adquisición se rechaza y se preserva el estado del sistema. Esta política evita borrar datos existentes para liberar espacio y prioriza la integridad de la información ya adquirida.

II.4. Requerimientos Funcionales y No Funcionales

Los requerimientos funcionales del sistema surgen de la necesidad de controlar una boya instrumental autónoma con múltiples subsistemas integrados. En términos generales, el sistema debe ser capaz de adquirir datos, validarlos, almacenarlos, procesarlos parcialmente y transmitir información seleccionada en forma remota.

Los requerimientos funcionales principales son:

- Adquirir mediciones ambientales de temperatura y humedad.
- Adquirir mediciones inerciales de aceleración y velocidad angular.
- Adquirir información de posicionamiento, tiempo y datos de navegación mediante AIS/GPS.

- Adquirir mediciones meteorológicas de velocidad y dirección del viento.
- Relevar variables eléctricas del sistema de alimentación, incluyendo parámetros asociados a panel solar, carga y batería.
- Ejecutar adquisiciones acústicas mediante placa de audio USB y almacenar archivos WAV con metadatos asociados.
- Procesar archivos de audio para obtener productos compactos compatibles con almacenamiento local y telemetría satelital.
- Registrar mediciones en archivos persistentes con formato estructurado y unidades explícitas.
- Registrar logs de ejecución, eventos relevantes, errores, advertencias y condiciones de diagnóstico.
- Transmitir mensajes de estado de sistema mediante Iridium SBD.
- Transmitir resultados seleccionados de procesamiento acústico mediante Iridium SBD.
- Ejecutar tareas periódicas mediante un planificador central configurable.
- Representar el comportamiento de cada módulo mediante Máquinas de Estados Finitos.
- Permitir ejecución con hardware real, con todos los módulos simulados o con simulación parcial de módulos seleccionados.
- Permitir pruebas automatizadas sin hardware y pruebas específicas contra hardware real.
- Mantener archivos de configuración editables para adaptar intervalos, rutas, parámetros de adquisición, habilitación de telemetría y uso de *mocks*.

Los requerimientos no funcionales responden a las restricciones propias de una plataforma autónoma embarcada. Entre ellos se destacan:

- **Autonomía operativa:** el sistema debe ejecutar ciclos de adquisición, almacenamiento y transmisión sin intervención humana continua.
- **Robustez:** el sistema debe tolerar fallas parciales de módulos, errores de lectura, indisponibilidad transitoria de periféricos y condiciones de comunicación no ideales.
- **Persistencia segura:** los datos adquiridos no deben perderse por fallas evitables de software, configuraciones ambiguas o políticas agresivas de liberación de espacio.
- **Bajo consumo relativo:** el software debe permitir ciclos de adquisición y transmisión compatibles con el presupuesto energético disponible.
- **Mantenibilidad:** la separación entre bajo nivel, FSM, Router y planificador debe facilitar la modificación de módulos individuales sin afectar la arquitectura completa.
- **Extensibilidad:** la arquitectura debe permitir la incorporación futura de nuevos sensores, nuevos productos de procesamiento o nuevos formatos de telemetría.
- **Verificabilidad:** el comportamiento del sistema debe poder probarse mediante tests automatizados, ejecución con *mocks*, ensayos de bajo nivel y pruebas de integración.
- **Trazabilidad:** las mediciones, eventos, configuraciones y resultados de prueba deben poder relacionarse con módulos, horarios de ejecución y condiciones operativas.

- **Degradación controlada:** ante una falla, el sistema debe registrar la condición, conservar datos previos, evitar acciones inseguras y continuar operando en la medida de lo posible.
- **Diagnóstico remoto:** los mensajes satelitales deben permitir inferir el estado general de la boya, incluyendo salud de módulos, almacenamiento, energía y continuidad operativa.

En conjunto, estos requerimientos justifican la arquitectura adoptada. La combinación de módulos de bajo nivel, FSM, Router, planificador central, almacenamiento estructurado, telemetría binaria, pruebas automatizadas y soporte de simulación permite abordar la complejidad de una boya operativa sin concentrar toda la lógica en un bloque monolítico. La Introducción Específica presentada en esta sección establece, por lo tanto, el marco técnico necesario para comprender las decisiones de diseño e implementación desarrolladas en las secciones siguientes.

III. Diseño e Implementación

III.1. Arquitectura del sistema

III.2. Proceso principal y enrutamiento de eventos

III.3. Planificador y gestión de tareas

III.4. Módulos de sensores

III.4.a. Sensores ambientales (AHT10, MPU6050)

III.4.b. Sensores meteorológicos y de posición

III.4.c. Procesamiento de audio

III.4.d. Gestión de energía

III.5. Telemetría satelital (Iridium SBD)

III.6. Protocolos y formatos de datos

III.7. Almacenamiento y políticas de persistencia

IV. Pruebas y Ensayos

IV.1. Pruebas unitarias y de integración

IV.2. Tests de bajo nivel y simulaciones (mocks)

IV.3. Ensayos operacionales en laboratorio

IV.4. Pruebas de campo y telemetría

IV.5. Resultados obtenidos

V. Conclusiones y Trabajo Futuro

V.1. Conclusiones

V.2. Logros alcanzados

V.3. Dificultades encontradas y lecciones aprendidas

V.4. Trabajo futuro

V.4.a. Mejoras propuestas

V.4.b. Nuevas funcionalidades

V.4.c. Escalabilidad y mantenimiento del sistema

REFERENCIAS

Bos, P. (2018). *Sistema de control para estación autónoma marítima de monitoreo de ruido ambiente*. Memoria del trabajo final, maestría en sistemas embebidos, Facultad de Ingeniería, Universidad de Buenos Aires, Ciudad Autónoma de Buenos Aires, Argentina. Director: Ariel Lutenberg.

Bos, P. (2026a). *boya*: Sistema python para adquisición, procesamiento y telemetría de una boya instrumental. Repositorio de software en GitHub. Repositorio del sistema embebido de control de la boya EAMMRA.

URL <https://github.com/patricciobos/boya>

Bos, P. (2026b). Documentación de periféricos del sistema *boya*. Carpeta `docs/` del repositorio `patricciobos/boya`. Incluye manuales y documentación técnica de periféricos: AHT10, Iridium, KBOX-S-J6412, LPS12-115, WindSonic y XTRA-N, entre otros.

URL <https://github.com/patricciobos/boya/tree/main/docs>

Bos, P., Cinquini, M., Prario, I., y Blanc, S. (2016). EAMMRA: Ingeniería conceptual. Informe Técnico AS 03/16, División Acústica Submarina, Dirección de Investigación de la Armada; UNIDEF (CONICET/MINDEF). Proyecto PIDDEF 32/14: EAMMRA.

Cinquini, M. (2026). Informe de caracterización de equipamiento de adquisición de señales acústicas. Nota Técnica AS 02/26, Proyecto PIDDEF 03/2022. Revisores: Igor Prario y Patricio Bos. Caracterización de placa Behringer UMC202HD y preamplificadores DAS-1/DAS-2 para la boya EAMMRA.

Cinquini, M., Bos, P., Prario, I., y Blanc, S. (2018). Diseño y fabricación de un preamplificador para hidrófonos. Informe Técnico AS 05/18, División Acústica Submarina, Dirección de Investigación de la Armada; UNIDEF (CONICET/MINDEF). Proyecto EAMMRA.

Cinquini, M., Bos, P., Prario, I., y Marques Rojo, R. (2019). Integración de subsistemas para el prototipo de EAMMRA. Informe Técnico AS 02/19, División Acústica Submarina, Dirección de Investigación de la Armada; UNIDEF (CONICET/MINDEF). Documento en etapa de redacción.

Ezcurra, H., Prario, I., Bos, P., Cinquini, M., y Blanc, S. (2019). Diseño de una boya prototipo para medición de nivel de ruido submarino en zona costera. Informe Técnico AS 01/19, División Acústica Submarina, Dirección de Investigación de la Armada; UNIDEF (CONICET/MINDEF). Proyecto PIDDEF 32/14: EAMMRA; consultoría técnica especializada de Ezcurra & Schmidt S.A.

pytest Development Team (2026). *pytest Documentation*. pytest-dev. Framework de pruebas para Python. URL <https://docs.pytest.org/en/stable/>

Python Packaging Authority (2026). *Install packages in a virtual environment using pip and venv*. Python Packaging Authority. Guía oficial de empaquetado e instalación de dependencias en Python. URL <https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/>

Python Software Foundation (2026). *venv — Creation of virtual environments*. Python Software Foundation. Documentación oficial de la biblioteca estándar de Python. URL <https://docs.python.org/3/library/venv.html>